

ALVA'S INSTITUTE OF ENGINEERING AND TECHNOLOGY

(Affiliated to Visvesvaraya Technological University, Belagavi)



A PROJECT THESIS ON

WeatherGPT

An AI-Powered Conversational Weather Intelligence Platform

with XGBoost Short-Term Prediction, LLM-Based Natural-Language Queries and Secure User Personalisation

Problem Statement: Weather data is abundant but scattered, technical and hard for ordinary users to act on; there is no single, conversational and locally-learning source of weather intelligence.

Submitted by

AYUSH C N

4AL23CD010

SHETTY VAISHNAVI SURESH

4AL23CD046

Under the guidance of

ANJIT RAJA

Department of Computer Science and Engineering (Data Science)

Alva's Institute of Engineering and Technology

Mijar, Moodbidri

Academic Year 2026 – 2027

CERTIFICATE

This is to certify that the project thesis entitled “WeatherGPT – An AI-Powered Conversational Weather Intelligence Platform” is a bonafide work carried out by Ayush C N (4AL23CD010), Shetty Vaishnavi Suresh (4AL23CD046) in partial fulfilment of the requirements of the degree for the academic year 2026–2027, under the supervision of the undersigned.

The thesis has been approved as it satisfies the academic requirements in respect of the project work prescribed for the said degree. The work embodied in this thesis has not been submitted to any other institution for the award of any degree or diploma.

Project Guide

Head of the Department

Principal

External Viva

Name of the Examiners

Signature with Date

1. _____

2. _____

DECLARATION

We, Ayush C N (4AL23CD010), Shetty Vaishnavi Suresh (4AL23CD046), hereby declare that the project thesis entitled “WeatherGPT – An AI-Powered Conversational Weather Intelligence Platform” is an original work carried out by us under the guidance of Anjit Raja, Department of Computer Science and Engineering (Data Science). This work has not been submitted, either in part or in full, to any other university or institution for the award of any degree, diploma or certificate.

We further declare that all sources of information, tools, libraries, datasets, application programming interfaces and references used during the course of this work have been duly acknowledged.

Place: Mijar

Date:

ACKNOWLEDGEMENT

The completion of this project would not have been possible without the guidance, encouragement and support of many people, and we take this opportunity to express our sincere gratitude to them.

We are grateful to Alva's Education Foundation and the management of Alva's Institute of Engineering and Technology, Mijar for providing an environment that encourages innovation and practical problem solving. We thank our Principal and the Head of the Department for their constant motivation and for making the laboratory and computing facilities available to us.

We express our deep sense of gratitude to our project guide Anjit Raja, Department of Computer Science and Engineering (Data Science), for the valuable suggestions, patient reviews and timely feedback that shaped this work from problem definition to final implementation. We also thank all the faculty members and non-teaching staff of the department for their cooperation.

We acknowledge the open-source communities and service providers whose tools made this work possible, in particular the Open-Meteo project for its freely accessible weather and geocoding interfaces, Groq for fast large-language-model inference, and the maintainers of FastAPI, SQLAlchemy, React, Vite and XGBoost.

Finally, we thank our parents, families and friends for their unwavering support and encouragement throughout this project.

[

Ayush C N

Shetty Vaishnavi Suresh

ABSTRACT

Weather information influences everyday decisions – commuting, travel, farming, events and safety – yet the data that describes it is spread across dashboards, numerical model outputs and technical products that most people cannot easily interpret. A user who simply wants to know “Will it rain in Bangalore tomorrow?” must find a source, read a table or chart, and reason about the result on their own.

This thesis presents WeatherGPT, a full-stack, AI-powered conversational weather intelligence platform. It combines real-time weather data from Open-Meteo (built on the ECMWF IFS model), locally trained XGBoost machine-learning models for temperature and precipitation prediction at 1, 3, 6, 12 and 24 hour horizons, and a Groq-hosted large language model (llama-3.3-70b-versatile) that interprets natural-language questions and phrases answers in plain language.

The back end is a Python FastAPI service with SQLAlchemy persistence on MariaDB/MySQL, JWT access and refresh tokens, and Argon2id password hashing. The front end is a React 18 and TypeScript single-page application built with Vite, with interactive Chart.js visualisations, location autocomplete, a protected dashboard and a chat interface with session support. Authenticated users can save locations, keep chat history, review search history and store preferences. The platform is containerised with Docker and Docker Compose.

The thesis documents the requirements, architecture, thirteen-table relational data model, REST API, security design, ML pipeline and LLM integration, and describes the testing approach. On the synthetic training data currently used, the temperature model reaches a mean absolute error of about 1.79 °C and R^2 of about 0.71, and the rain classifier reaches about 85% accuracy with a low F1 score of about 0.125 owing to class imbalance. These figures are reported candidly as a baseline; the thesis also specifies how the models can be retrained on real observations. Known limitations – a stubbed IMD alert service, missing rate limiting and partial refresh-token rotation – are analysed together with a deployment checklist and future scope.

Keywords: Conversational AI, weather forecasting, FastAPI, React, Open-Meteo, XGBoost, large language model, Groq, JWT, Argon2, MariaDB.

TABLE OF CONTENTS

CERTIFICATE	2
DECLARATION	3
ACKNOWLEDGEMENT	4
ABSTRACT	5
LIST OF ABBREVIATIONS	8
CHAPTER 1: INTRODUCTION	9
1.1 Background	9
1.2 Problem Statement	9
1.3 Motivation.....	9
1.4 Objectives	9
1.5 Scope of the Project	10
1.6 Significance of the Work	10
1.7 Organisation of the Thesis	10
CHAPTER 2: LITERATURE REVIEW	12
2.1 Overview of Weather Forecasting	12
2.2 Delivery of Weather Information.....	12
2.3 Machine Learning for Short-Term Prediction	12
2.4 Large Language Models as Domain Assistants	13
2.5 Relevant Technologies.....	13
2.6 Research Gap	13
2.7 Summary	13
CHAPTER 3: REQUIREMENT ANALYSIS	14
3.1 Stakeholders.....	14
3.2 Functional Requirements	14
3.3 Non-Functional Requirements	15
3.4 Use Case Description.....	15
3.5 Feasibility Study	15
3.6 Constraints and Assumptions.....	16
CHAPTER 4: SYSTEM DESIGN	17
4.1 Design Goals	17
4.2 System Architecture.....	17
4.3 Module Decomposition.....	17
4.4 Data Model.....	18
4.5 Conversational Query Workflow	18
4.6 REST API Design	19
4.7 Security Design.....	19
4.8 LLM and ML Integration Design	20
4.9 User Interface Design	20
4.10 Summary	20
CHAPTER 5: IMPLEMENTATION	21
5.1 Technology Stack.....	21
5.2 Project Structure.....	21
5.3 Environment Configuration	21
5.4 Authentication and Authorisation	22
5.5 Weather Service	23
5.6 LLM Service	23
5.7 ML Service.....	23
5.8 Chat Module.....	23
5.9 User Personalisation.....	24
5.10 Front-End Implementation.....	24

5.11 Database Initialisation and Containers.....	24
5.12 Running the Application.....	24
CHAPTER 6: ML, LLM AND ANALYTICS	25
6.1 Objective of the ML Layer	25
6.2 Input Features.....	25
6.3 Models.....	25
6.4 Training Pipeline.....	26
6.5 Evaluation	26
6.6 Grounding the LLM.....	26
6.7 Roadmap for Improving the Models.....	26
6.8 Analytics Presented to Users	27
6.9 Summary	27
CHAPTER 7: TESTING AND VALIDATION	28
7.1 Testing Strategy	28
7.2 Test Environment.....	28
7.3 Test Cases	28
7.4 Execution and Results.....	29
7.5 Known Issues and Resolutions	29
7.6 Summary	30
CHAPTER 8: RESULTS AND DISCUSSION.....	31
8.1 Achievement of Objectives.....	31
8.2 Feature Summary of the Working System.....	31
8.3 Analysis of the ML Metrics	31
8.4 Discussion	32
8.5 Social and Economic Impact	32
8.6 Deployment Considerations.....	32
CHAPTER 9: CONCLUSION AND FUTURE SCOPE.....	34
9.1 Conclusion	34
9.2 Future Scope	34
9.3 Closing Remarks.....	34
REFERENCES.....	35
APPENDIX A: INSTALLATION AND EXECUTION GUIDE	36
A.1 Prerequisites	36
A.2 Steps.....	36
A.3 Access URLs.....	36
APPENDIX B: SAMPLE API REQUESTS AND RESPONSES.....	37
B.1 Login	37
B.2 Current weather	37
B.3 Chat	37
APPENDIX C: TEST EXECUTION RECORD.....	38
APPENDIX D: DOCKER COMPOSE CONFIGURATION	39

LIST OF ABBREVIATIONS

Abbreviation	Expansion
AI	Artificial Intelligence
API	Application Programming Interface
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
ECMWF	European Centre for Medium-Range Weather Forecasts
HTTP(S)	HyperText Transfer Protocol (Secure)
IFS	Integrated Forecasting System
IMD	India Meteorological Department
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
MAE	Mean Absolute Error
ML	Machine Learning
NLP	Natural Language Processing
NWP	Numerical Weather Prediction
ORM	Object Relational Mapping
REST	Representational State Transfer
RMSE	Root Mean Squared Error
R ²	Coefficient of Determination
SPA	Single-Page Application
SQL	Structured Query Language
TTL	Time To Live
UI	User Interface
XGBoost	Extreme Gradient Boosting

CHAPTER 1: INTRODUCTION

1.1 Background

Weather shapes almost every outdoor activity and many economic ones. Farmers schedule irrigation, spraying and harvest around rainfall; commuters and logistics operators plan around storms and heat; event organisers and ordinary households simply want to know whether to carry an umbrella. Modern forecasting rests on numerical weather prediction (NWP), in which supercomputers integrate the equations of atmospheric physics from a measured initial state. Global systems such as the ECMWF Integrated Forecasting System (IFS) produce high-quality forecasts that are now openly reachable through programmatic interfaces such as Open-Meteo.

Access to data, however, is not the same as access to understanding. Raw forecasts arrive as many hourly variables – temperature, humidity, pressure, wind, cloud cover, precipitation – encoded with numeric weather codes. Turning them into an answer to a concrete question requires effort and some meteorological literacy. At the same time, large language models (LLMs) have made it practical for software to accept free-form questions and reply in fluent language, provided their answers are anchored to real data rather than generated from memory.

WeatherGPT brings these strands together. It retrieves live forecasts, augments them with locally trained machine-learning predictions, and uses an LLM as a conversational front door, all behind a secure, personalised web application.

1.2 Problem Statement

The problem statement addressed by this project is: “Weather data is abundant but scattered, technical and hard for ordinary users to act on; there is no single, conversational and locally-learning source of weather intelligence.” The expected solution is a web platform that lets a user ask weather questions in natural language, receive answers grounded in real forecast data, view current conditions, forecasts and history in one dashboard, and see short-horizon predictions from models the platform trains itself.

The expected benefits are easier access to weather information for non-experts, faster decisions, personalised locations and history, and a data-driven foundation on which better local prediction can be built. This thesis documents the design and implementation of WeatherGPT and evaluates it honestly against those goals.

1.3 Motivation

Several observations motivated this work. First, most weather applications present numbers and icons but cannot answer follow-up questions such as “Is tomorrow better than today for spraying?”. Second, general-purpose chatbots can discuss weather fluently but, without a live data connection, they may state outdated or invented conditions. Third, global forecast models are coarse and may be improved locally by lightweight machine-learning models trained on observed data. Fourth, many public weather tools require no account, and therefore cannot remember a user’s places or conversations.

WeatherGPT responds to these gaps with a grounded conversational interface, an integrated ML prediction layer, and secure user accounts with saved locations, chat sessions and preferences.

1.4 Objectives

The objectives of the project are:

- To build a full-stack web application that presents current weather, hourly and daily forecasts, and historical analysis for any searchable location.

- To integrate a large language model so that users can ask weather questions in natural language and receive answers derived from retrieved data.
- To train and serve XGBoost models that predict temperature and probability of rain at multiple short horizons.
- To provide secure registration, login and session handling using JWT access and refresh tokens and Argon2id password hashing.
- To support personalisation through saved locations, chat sessions, search history and preferences.
- To design a normalised relational schema that caches weather data and records queries, predictions and alerts.
- To package the system for reproducible deployment with Docker and to document its limitations and a production checklist.

1.5 Scope of the Project

WeatherGPT covers the complete flow from a user question to a data-backed answer: registration and login, location search and geocoding, retrieval of current, forecast and historical weather, ML prediction at five horizons, conversational querying with stored sessions, and dashboard visualisation with charts and alert panels.

The following are outside the present scope: a native mobile application, live integration with official India Meteorological Department (IMD) alert feeds (currently a stub), radar or satellite imagery, production-grade rate limiting, and models trained on a long real observation record. These are discussed as future enhancements in Chapter 9.

1.6 Significance of the Work

- **For everyday users:** a single place to ask weather questions and read answers in plain language.
- **For frequent users:** saved locations, history and preferences that persist across sessions.
- **For students and developers:** a documented reference architecture that combines an external data API, a local ML model and an LLM in one application.
- **For future research:** a stored record of observations, predictions and queries that can be used to retrain and evaluate models.

1.7 Organisation of the Thesis

Table 1.1: Organisation of the thesis

Chapter	Title	Summary
1	Introduction	Background, problem statement, objectives, scope and significance.
2	Literature Review	Forecasting, delivery of weather information, ML and LLM approaches, technologies and research gap.
3	Requirement Analysis	Stakeholders, functional and non-functional requirements, feasibility.
4	System Design	Architecture, data model, API, security and interface design.

Chapter	Title	Summary
5	Implementation	Technology stack, project structure, configuration and module walkthrough.
6	ML, LLM and Analytics	Feature set, training pipeline, evaluation and LLM grounding.
7	Testing and Validation	Test strategy, test cases, known issues and resolutions.
8	Results and Discussion	Outcomes against objectives, metric analysis, limitations, SWOT.
9	Conclusion and Future Scope	Summary and directions for further work.

CHAPTER 2: LITERATURE REVIEW

2.1 Overview of Weather Forecasting

Operational weather forecasting is dominated by numerical weather prediction. Centres such as ECMWF run global models that assimilate observations from satellites, radiosondes, aircraft and surface stations to form an initial state, then integrate the governing equations forward in time. The steady improvement in skill of such systems over recent decades is well documented in the literature. The ECMWF IFS is widely regarded as among the most accurate global medium-range systems, and its output can be accessed by developers through open interfaces.

Two limitations are relevant to application builders. First, global models have finite resolution and cannot resolve every local effect, so location-specific bias is common. Second, model output is delivered as gridded variables and codes rather than as advice. Post-processing – statistical or machine-learning correction of model output for a location – is therefore a well-established way to add value, and presentation layers are needed to make the output usable.

2.2 Delivery of Weather Information

Weather reaches the public through national meteorological services, commercial applications, television and social media. National services such as the India Meteorological Department provide authoritative forecasts and warnings, often in formats aimed at professionals. Consumer applications present clean current-conditions views but usually offer fixed screens rather than dialogue. Table 2.1 summarises typical approaches and their limitations.

Table 2.1: Comparison of approaches to delivering weather information

Approach	Strengths	Limitations
National meteorological portals	Authoritative data and official warnings	Technical presentation; limited conversational access
Consumer weather apps	Polished current-conditions and forecast views	Fixed screens; no natural-language follow-up; opaque models
General-purpose chatbots	Fluent natural-language interaction	May lack live data and can state invented conditions
Open weather APIs	Free, structured, programmatic access	Require developers to build the interface and interpretation
WeatherGPT	Live data + local ML + LLM dialogue + user accounts	ML currently trained on synthetic data; alert service is a stub

2.3 Machine Learning for Short-Term Prediction

Gradient-boosted decision trees, of which XGBoost is a leading implementation, are effective on tabular data with mixed numeric features and modest sample sizes. For short-horizon weather prediction, useful inputs include current conditions, time-of-day and seasonal indicators, and lagged values that capture recent trend. Regression models estimate continuous quantities such as temperature, while classifiers estimate the probability of an event such as rain.

Evaluating rain classifiers needs care. Rain is a minority event at most locations, so overall accuracy can look high even for a model that seldom predicts rain. Metrics that respect class imbalance – precision, recall and F1 score for the rain class, or precision–recall curves – give a more honest picture. Section 6.5 applies this caution to the results of this project.

2.4 Large Language Models as Domain Assistants

LLMs can classify intent, extract entities such as place names and dates from free text, and compose fluent answers. Their weakness in factual domains is hallucination: without access to current data, a model will still produce plausible-sounding but unfounded statements. The prevailing remedy is grounding – retrieving facts from a trusted source and instructing the model to phrase only what those facts support. WeatherGPT follows this pattern, using the LLM for interpretation and phrasing while numbers come from the weather service and ML models.

2.5 Relevant Technologies

2.5.1 Web API framework

FastAPI is an asynchronous Python framework built on Starlette and Pydantic. It validates requests against typed schemas, generates interactive OpenAPI documentation automatically, and is well suited to services that call several external APIs concurrently.

2.5.2 Object-relational mapping

SQLAlchemy 2.x maps Python classes to relational tables and abstracts database differences. Combined with PyMySQL it targets MariaDB/MySQL, and the same models can in principle move to another supported database.

2.5.3 Token-based authentication and password hashing

JSON Web Tokens (RFC 7519) carry signed claims that a server can verify without a session store, which suits stateless REST APIs. Short-lived access tokens paired with longer-lived refresh tokens limit the damage of token theft. Argon2, the winner of the Password Hashing Competition and specified in RFC 9106, is a memory-hard password hashing function designed to resist GPU-based cracking; Argon2id combines data-dependent and data-independent memory access.

2.5.4 Weather data and geocoding

Open-Meteo offers free, key-less access to forecasts derived from several national weather models, including ECMWF IFS, together with a geocoding interface that resolves place names to coordinates. Its JSON responses make it convenient for prototypes and academic work.

2.5.5 Front-end stack

React organises a user interface into composable components; TypeScript adds static types that catch interface mismatches early; Vite provides fast development builds and a proxy to the back end. Chart.js renders interactive charts in the browser.

2.6 Research Gap

From the review, three gaps emerge. First, few open projects combine a live NWP data source, a locally trained prediction layer and a conversational interface in one deployable package. Second, conversational weather tools often lack persistent user accounts, so they cannot remember locations or history. Third, ML components in student and prototype projects are seldom documented with their data provenance and limitations. WeatherGPT addresses these gaps by delivering an integrated, containerised platform and by reporting the state of its models transparently.

2.7 Summary

This chapter reviewed forecasting, the delivery of weather information, machine-learning and LLM approaches, and the technologies used. The research gap motivates the requirement analysis presented next.

CHAPTER 3: REQUIREMENT ANALYSIS

3.1 Stakeholders

Requirement analysis began by identifying the people who interact with, or are affected by, the system. Each has distinct goals and technical comfort, which drove many design decisions.

Table 3.1: Stakeholders and needs

Stakeholder	Goals	Key needs
Casual user	Quickly learn the weather for a place	Search, plain-language answers, no mandatory sign-up for chat
Registered user	Track favourite places and past questions	Saved locations, chat history, preferences, secure login
Field / outdoor planner	Plan activities around rain and temperature	Hourly forecast, rain probability, short-horizon predictions
Developer / administrator	Operate and improve the platform	Docs, health checks, containerisation, model retraining

3.2 Functional Requirements

Table 3.2: Functional requirements

ID	Requirement	Actor
FR-01	Register and log in with e-mail and password	User
FR-02	Issue JWT access and refresh tokens; refresh and logout	System
FR-03	View and update profile; change password	Registered user
FR-04	Request and complete a password reset using a token	User
FR-05	Search locations by name with autocomplete geocoding	User
FR-06	Show current weather observation for a location	User
FR-07	Show 7-day forecast with hourly and daily breakdown	User
FR-08	Show historical weather analysis (default 10 years)	User
FR-09	Display active weather alerts for a location	User
FR-10	Provide ML predictions of temperature and rain at 1, 3, 6, 12 and 24 h	System
FR-11	Answer natural-language weather questions, returning intent and entities	User / System
FR-12	Allow chat without login; store sessions and history for logged-in users	User
FR-13	Create, list, update and delete saved locations; set a default	Registered user
FR-14	View and clear search history	Registered user
FR-15	Read and update user preferences	Registered user
FR-16	Visualise temperature, rainfall and monthly statistics as charts	User
FR-17	Train temperature and rain models from stored observations	Developer
FR-18	Run the full stack with Docker / Docker Compose	Developer

3.3 Non-Functional Requirements

Table 3.3: Non-functional requirements

Category	Requirement
Security	Argon2id password hashing; HS256-signed JWTs; protected routes; secrets held in environment variables; HttpOnly refresh cookie
Performance	Chat responses recorded with response time; external calls made asynchronously; cached locations reduce repeated geocoding
Usability	Responsive interface; autocomplete search; consistent cards and charts; clear error states
Reliability	Health-check endpoints; graceful behaviour when an external service is unavailable
Maintainability	Layered modules (API, services, models, schemas); typed front-end interfaces mirroring back-end schemas
Portability	Containerised with Docker; configuration through environment variables
Extensibility	Service layer isolates the weather provider, LLM and ML model so each can be replaced

3.4 Use Case Description

The primary flow is the conversational query: a user opens the dashboard, selects or searches a location, types a question, and receives an answer accompanied by the underlying weather data, predictions and alerts. Registered users additionally save locations and revisit earlier sessions.

Table 3.4: Principal use cases

Use case	Actor	Main flow
UC-1 Register / Login	User	Submit credentials → server verifies → access token and refresh cookie issued → dashboard
UC-2 View weather	User	Search place → select → current, forecast, charts and alerts load
UC-3 Ask a question	User	Type question → intent and entities extracted → data retrieved → answer returned
UC-4 Manage locations	Registered user	Save location → set default → edit or delete
UC-5 Review history	Registered user	Open sessions or search history → read or clear
UC-6 Retrain models	Developer	Populate observations → run training script → new artefacts and metrics

3.5 Feasibility Study

3.5.1 Technical feasibility

All components use mature open-source technologies (Python, FastAPI, SQLAlchemy, React, XGBoost) and accessible services (Open-Meteo without a key, Groq with a free API key). The application runs on an ordinary laptop, so technical risk is low.

3.5.2 Economic feasibility

Development tools are free. Open-Meteo is free for non-commercial use, and Groq offers a free tier. Costs arise only on scaling – hosting, a managed database and higher LLM usage – which are modest for a prototype.

3.5.3 Operational feasibility

The interface is browser-based and needs no installation by end users. Operating the platform requires a database, an API key and periodic model retraining, all of which are documented in the appendices.

3.6 Constraints and Assumptions

- Users have a device with a browser and internet access.
- A Groq API key and a JWT signing secret of at least 32 characters are supplied by the deployer.
- Python 3.12 is required for compatibility with the pinned dependencies (pydantic-core).
- Prediction quality depends on the amount and realism of stored observations; the present models are trained on synthetic data.
- Official IMD alert integration requires credentials that are not part of this project.

CHAPTER 4: SYSTEM DESIGN

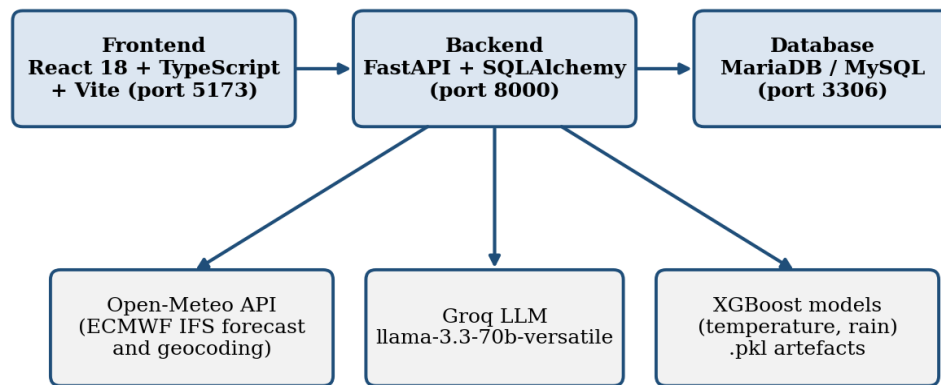
4.1 Design Goals

The design follows four goals: separate presentation, application logic and data; keep external services (weather, LLM, ML) behind replaceable service modules; treat authentication and secret handling as first-class concerns; and keep the system simple enough to run locally and in containers.

4.2 System Architecture

WeatherGPT uses a layered client–server architecture. The presentation layer is a React 18 single-page application served by Vite on port 5173 during development. It calls a JSON REST API implemented in FastAPI on port 8000 under the /api prefix. The application layer contains service modules for authentication, weather retrieval, LLM interaction, ML inference and user data. The data layer uses SQLAlchemy over MariaDB/MySQL on port 3306. Open-Meteo, Groq and the local XGBoost artefacts are accessed only through service modules.

Layered architecture of WeatherGPT



External services and local ML layer

Figure 4.1: Layered architecture of WeatherGPT

4.3 Module Decomposition

Table 4.1: Module decomposition

Module	Responsibility
api/auth.py	Registration, login, refresh, logout, profile, password change and reset, saved locations, sessions, search history, preferences
api/weather.py	Current, forecast, historical, alerts, location search and health endpoints
api/chat.py	Conversational endpoint, chat history and session listing
api/deps.py	FastAPI dependencies for authentication and database sessions
services/auth_service.py	User authentication, Argon2 hashing and token creation
services/weather_service.py	Open-Meteo client and IMD alerts stub
services/llm_service.py	Groq integration for intent, response generation and advisories
services/ml_service.py	XGBoost inference and training utilities
services/database_service.py	CRUD for weather and user entities

Module	Responsibility
services/user_service.py	Saved locations, chat sessions, search history and preferences
ml/train.py	Training script: fetch data, train models, save artefacts and metrics

4.4 Data Model

The relational schema, created through SQLAlchemy's `create_all`, centres on users, their locations and chat activity, and on cached weather data. A user owns many saved locations, chat sessions and search-history entries; a chat session contains many messages. Weather observations, forecasts, predictions and alerts are linked to canonical location records so that data fetched once can be reused and later used for training.

Table 4.2: Principal database tables

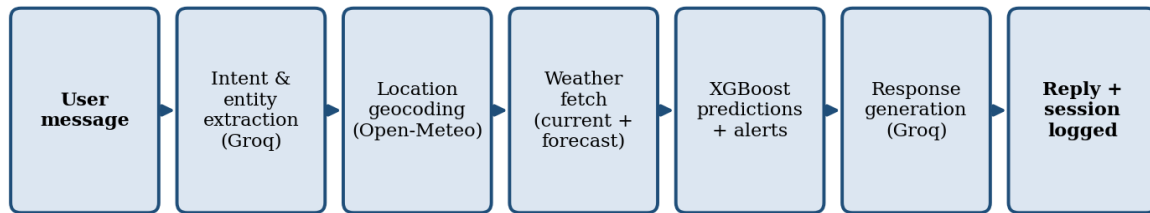
Table	Purpose
users	User accounts, credentials and preferences
user_locations	Saved locations per user
chat_sessions	Conversation sessions
chat_messages	Individual messages with weather context
search_history	Location search history
locations	Canonical weather locations cached from the API
weather_observations	Current weather observations (training source)
weather_forecasts	Forecast data, hourly and daily
weather_predictions	ML model predictions
weather_alerts	Weather alerts (IMD and generated)
user_queries	Chat query log with intent and entities
historical_weather	Yearly and monthly historical aggregates

The project documentation reports thirteen tables in total; Table 4.2 lists the twelve principal entities described there, and the full definition is in `backend/models/__init__.py`.

4.5 Conversational Query Workflow

A chat request carries the user's message, an optional session identifier, optional coordinates and location name, and a language code. The response contains the generated text, the detected intent, extracted entities, the weather data used, ML predictions, any alerts, the session identifier and the response time in milliseconds. This structure shows the intended workflow: interpret the question, resolve the location, gather weather and predictions, then phrase an answer from that data.

Conversational query workflow



The exchange is persisted in the chat session and query-log tables.

Figure 4.2: Conversational query workflow

4.6 REST API Design

The API follows REST conventions with JSON payloads and bearer-token authentication. All endpoints are prefixed with /api. Table 4.3 lists the endpoint groups; authentication is shown as required, optional or none.

Table 4.3: REST API overview

Group	Representative endpoints	Auth
Authentication	POST /auth/register, /auth/login, /auth/refresh, /auth/logout, /auth/forgot-password, /auth/reset-password	None
Profile	GET/PATCH /auth/me; POST /auth/change-password	Required
Weather	GET /weather/current, /weather/forecast, /weather/historical, /weather/alerts, /weather/locations/search, /weather/health	None
Chat	POST /chat; GET /chat/history/{session_id}	Optional
Chat sessions	GET /chat/sessions; GET/POST /auth/chat/sessions; GET /auth/chat/sessions/{id}	Required
Locations	GET/POST /auth/locations; GET /auth/locations/default; PATCH/DELETE /auth/locations/{id}	Required
History & preferences	GET/DELETE /auth/search-history; GET/PATCH /auth/preferences	Required

4.7 Security Design

- **Password storage:** Argon2id with memory cost 102400, time cost 2 and parallelism 8; bcrypt retained as a fallback.
- **Tokens:** HS256-signed JWTs; access token valid for 30 minutes and sent as a Bearer header; refresh token valid for 7 days and held in an HttpOnly cookie.
- **Secrets:** database URL, Groq key and signing key are read from environment variables (.env) and excluded from version control.
- **Input validation:** Pydantic schemas validate every request body; SQLAlchemy parameterisation prevents SQL injection.
- **Cross-origin control:** CORS origins are configured explicitly (localhost:5173 and localhost:3000 by default).

- **Password reset:** a reset token is issued on request; in development it is returned in the response for convenience and must be delivered by e-mail in production.

4.8 LLM and ML Integration Design

The LLM service uses Groq’s llama-3.3-70b-versatile model for three tasks: identifying the user’s intent, generating a natural-language response, and producing advisories. The ML service loads two pickled XGBoost artefacts at start-up and returns predictions for five horizons. Both services are optional at run time: if models are absent, weather retrieval still works, and the design keeps numbers under the control of the weather and ML services rather than the LLM.

4.9 User Interface Design

The front end is a React 18 and TypeScript single-page application. Public routes are the landing, login, registration and password-reset pages; the dashboard is protected by a ProtectedRoute component that redirects unauthenticated users to the login page. The dashboard composes a location search box with autocomplete, a current-weather card, hourly and daily forecast components, chart panels for temperature, rainfall and monthly statistics, an alerts panel, prediction and statistics summaries, and a chat interface with session support.

4.10 Summary

This chapter presented the architecture, modules, data model, workflow, API and security design of WeatherGPT. The next chapter describes how this design was implemented.

CHAPTER 5: IMPLEMENTATION

5.1 Technology Stack

Table 5.1: Technology stack

Layer	Technology	Version
Backend framework	FastAPI (served by Uvicorn)	0.115.0
Frontend	React + TypeScript + Vite	18.3.1 / 5.6.2
Database	MariaDB / MySQL via PyMySQL	1.1.1
ORM	SQLAlchemy	2.0.36
LLM	Groq (llama-3.3-70b-versatile)	groq 0.13.0
Machine learning	XGBoost + scikit-learn	2.1.4 / 1.7.2
Authentication	JWT (HS256) via python-jose; Argon2 via argon2-cffi	—
Weather API	Open-Meteo (ECMWF IFS) via httpx	httpx 0.27.2
Charts	Chart.js wrappers	—
Containers	Docker and Docker Compose (MariaDB 10.11)	—

FastAPI was chosen for typed request validation and automatic API documentation, SQLAlchemy for database-agnostic persistence, and React with TypeScript for a maintainable interface whose types mirror the back-end schemas. Each choice trades some simplicity for structure, which suits a project intended to be extended and reproduced.

5.2 Project Structure

The repository separates the back end, the front end and the machine-learning code. The layout below summarises the organisation used in the project.

```

weathergpt-web/
├── backend/
│   ├── api/           # auth.py, chat.py, deps.py, weather.py
│   ├── models/        # SQLAlchemy models
│   ├── services/      # auth, database, llm, ml, user, weather
│   ├── ml/train.py    # training script and trained_models/
│   ├── config.py      # Pydantic settings (env-driven)
│   ├── database.py    # engine, session, init_db
│   ├── schemas.py     # request / response schemas
│   ├── main.py        # app, lifespan, CORS, routers
│   ├── Dockerfile, requirements.txt, .env.example, init.sql
├── frontend/
│   ├── src/ components/ hooks/ pages/ services/ types/ utils/
├── ml/                # standalone training entry point
├── test_*.py          # manual test scripts
├── docker-compose.yml, start.sh, README.md

```

5.3 Environment Configuration

All secrets and deployment-specific settings are read from a .env file through Pydantic Settings, so no credential is hard-coded. Table 5.2 lists the main variables and their defaults.

Table 5.2: Backend environment variables

Variable	Default / note
DATABASE_URL	mysql+pymysql://weathergpt:weathergpt@127.0.0.1:3306/weathergpt

Variable	Default / note
GROQ_API_KEY	Required; obtained from console.groq.com
GROQ_MODEL	llama-3.3-70b-versatile
OPEN_METEO_BASE_URL	https://api.open-meteo.com/v1
OPEN_METEO_GEOCODING_URL	https://geocoding-api.open-meteo.com/v1
IMD_API_KEY	Optional; the IMD service is currently a stub
MODEL_DIR	./ml/trained_models
TEMPERATURE_MODEL_PATH / RAIN_MODEL_PATH	temperature_model.pkl / rain_model.pkl
SECRET_KEY	Required; at least 32 characters
ALGORITHM	HS256
ACCESS_TOKEN_EXPIRE_MINUTES	30
REFRESH_TOKEN_EXPIRE_DAYS	7
PASSWORD_HASH_ALGORITHM	argon2
CORS_ORIGINS	http://localhost:5173, http://localhost:3000

The front end reads a single variable from frontend/.env:

```
VITE_API_URL=http://localhost:8000/api
```

5.4 Authentication and Authorisation

Registration stores an Argon2id hash of the password. On login the server verifies the hash and returns an access token in the response body and sets the refresh token as an HttpOnly cookie. Protected routes use a FastAPI dependency (deps.py) that validates the bearer token and loads the current user. The excerpt below is an illustrative sketch of the approach, not a verbatim copy of the source.

```
from argon2 import PasswordHasher
from jose import jwt
from datetime import datetime, timedelta, timezone

ph = PasswordHasher(time_cost=2, memory_cost=102400, parallelism=8)

def hash_password(plain: str) -> str:
    return ph.hash(plain)

def create_token(sub: str, minutes: int, secret: str) -> str:
    exp = datetime.now(timezone.utc) + timedelta(minutes=minutes)
    return jwt.encode({"sub": sub, "exp": exp}, secret, algorithm="HS256")
```

Because the access token is short-lived (30 minutes) and the refresh token is confined to an HttpOnly cookie that scripts cannot read, the exposure from cross-site scripting is reduced. The documentation notes that refresh-token rotation is partial: a new refresh token is issued on refresh but the old one is not explicitly invalidated (see Section 7.5).

5.5 Weather Service

weather_service.py wraps the Open-Meteo forecast and geocoding interfaces using asynchronous httpx calls. The current-weather endpoint accepts latitude, longitude and an optional location name and returns the observation, forecast, alerts and ML predictions in a single payload, so that the front end can

render a full dashboard with one request. The forecast endpoint returns seven days of hourly and daily data; the historical endpoint returns yearly and monthly aggregates, ten years by default; and the location-search endpoint returns geocoding candidates for the autocomplete box. Weather codes are mapped to human-readable descriptions such as “Partly cloudy”.

```
# Illustrative sketch of an Open-Meteo request
async def fetch_forecast(client, lat, lon):
    r = await client.get(f"{OPEN_METEO_BASE_URL}/forecast", params={
        "latitude": lat, "longitude": lon,
        "current": "temperature_2m,relative_humidity_2m,weather_code",
        "hourly": "temperature_2m,precipitation_probability",
        "daily": "temperature_2m_max,temperature_2m_min,precipitation_sum",
        "timezone": "auto"})
    r.raise_for_status()
    return r.json()
```

The `IMDAAlertService` class is currently a stub. It defines the place where official India Meteorological Department alerts will be integrated once credentials are available, and the alerts field of each response is therefore empty unless generated alerts are produced.

5.6 LLM Service

`llm_service.py` integrates the Groq client with the llama-3.3-70b-versatile model. It provides three capabilities: detecting the intent of a message (for example, `rain_probability`), generating the reply text from the retrieved data, and producing advisory text. The chat endpoint returns the intent, extracted entities, supporting weather data and predictions along with the reply so that the interface can display the evidence next to the sentence. The design keeps factual values in the weather and ML services and asks the LLM only to interpret and phrase them (Section 6.6).

5.7 ML Service

`ml_service.py` loads the pickled XGBoost artefacts from `MODEL_DIR` and computes the eighteen input features from the latest observation and recent history. It returns predictions for temperature and rain at horizons of 1, 3, 6, 12 and 24 hours. It also contains the training utilities used by `ml/train.py`. Training and evaluation are described in Chapter 6.

5.8 Chat Module

The chat endpoint accepts a message, an optional session identifier, optional coordinates and location name, and a language code. Authentication is optional: anonymous users can chat, while logged-in users have sessions and messages persisted and can list them later. Each exchange is also written to the `user_queries` table together with the detected intent and entities, and the response time is measured and returned. The sketch below shows the intended sequence.

```
# Illustrative sketch of the chat flow
async def chat(req, user=None):
    intent, entities = await llm.classify(req.message)
    place = resolve_location(entities, req)          # geocode if needed
    weather = await weather_service.get(place)      # Open-Meteo
    preds = ml_service.predict(weather)             # XGBoost
    alerts = await imd_service.get_alerts(place)    # stub today
    text = await llm.respond(req.message, weather, preds, alerts)
    save_session_and_query(user, req, intent, entities, text)
    return {"response": text, "intent": intent, "entities": entities,
            "weather_data": weather, "predictions": preds, "alerts": alerts}
```

5.9 User Personalisation

Registered users can save locations, mark one as default, edit or delete them; review and clear their search history; and read or update preferences. Chat sessions can be listed and reopened with their messages. These features are exposed through the `/auth/*` endpoints and are implemented in `user_service.py`, so the weather and chat modules remain independent of account management.

5.10 Front-End Implementation

The React application is organised into pages, components, hooks, services and types. `useAuth.tsx` provides an authentication context and token management; `api.ts` wraps the back-end endpoints with an Axios client and typed functions; and the `types` folder mirrors the Pydantic schemas so that mismatches are caught at compile time. `Charts.tsx` contains Chart.js wrappers for temperature, rainfall and monthly statistics. `ProtectedRoute` redirects unauthenticated users to the login page. During development Vite proxies `/api` to the back end at port 8000.

[Insert screenshot: WeatherGPT dashboard with current weather, forecast and charts]

Figure 5.1: WeatherGPT dashboard (replace with a screenshot of the running application)

[Insert screenshot: chat interface answering "Will it rain in Bangalore tomorrow?"]

Figure 5.2: Conversational chat interface (replace with a screenshot)

5.11 Database Initialisation and Containers

Tables are created with SQLAlchemy's `Base.metadata.create_all`, invoked through `init_db`, and an `init.sql` file is provided for database initialisation. The Dockerfile in the backend directory builds the API image, and the documented Compose configuration defines three services – a MariaDB 10.11 database with a persistent volume, the backend on port 8000 and the frontend on port 5173 – with the database URL, Groq key and secret key passed as environment variables (Appendix D).

5.12 Running the Application

1. Create a Python 3.12 virtual environment and install dependencies: `pip install -r requirements.txt`.
2. Copy `.env.example` to `.env` and set `DATABASE_URL`, `GROQ_API_KEY` and `SECRET_KEY` (Section 5.3).
3. Create the tables: `python -c "from backend.database import init_db; init_db()"`.
4. Start the API: `python -m uvicorn backend.main:app --host 0.0.0.0 --port 8000 --reload`.
5. In the frontend folder run `npm install` and `npm run dev`, then open `http://localhost:5173`.
6. Optionally train the ML models with `python ml/train.py`, and run the tests with `python -m pytest -v`.

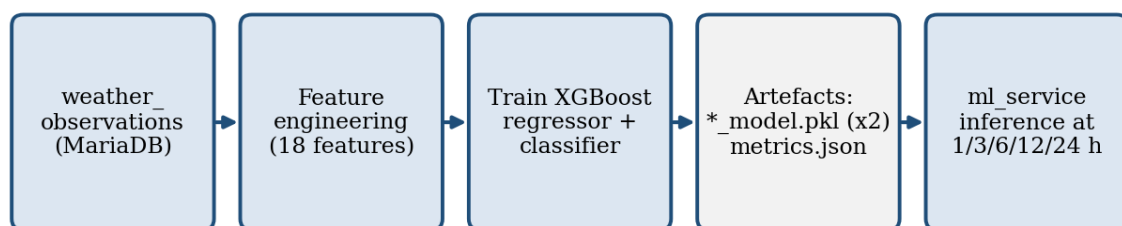
CHAPTER 6: ML, LLM AND ANALYTICS

This chapter describes the machine-learning models, the way the LLM is grounded in retrieved data, and the analytics presented to users. It reports the model metrics exactly as documented for the current build and is explicit about their limits.

6.1 Objective of the ML Layer

The ML layer estimates two quantities at five horizons (1, 3, 6, 12 and 24 hours): the air temperature and the probability of rain. These estimates complement the forecast returned by Open-Meteo and give the platform a component it can retrain on its own accumulated observations.

Machine-learning training and inference pipeline



If real observations are insufficient, the training script generates synthetic data.

Figure 6.1: Machine-learning training and inference pipeline

6.2 Input Features

Eighteen features are used, grouped as current conditions, time indicators and lagged values.

Table 6.1: Model input features

Group	Features
Current weather (8)	temperature, humidity, pressure, wind_speed, wind_direction, cloud_cover, rainfall, visibility
Time (5)	hour, day_of_year, month, day_of_week, is_night
Lag (5)	temp_1h_ago, temp_3h_avg, rain_3h_sum, temp_24h_ago, rain_24h_sum

6.3 Models

Table 6.2: Models

Target	Algorithm	Output
Temperature	XGBoost Regressor	Predicted temperature (°C) at 1, 3, 6, 12, 24 h
Rain	XGBoost Classifier	Probability of precipitation at 1, 3, 6, 12, 24 h

6.4 Training Pipeline

Training is run with `python ml/train.py` and requires the `weather_observations` table to be populated. The script performs the following steps: fetches historical observations from the database; generates

synthetic data if insufficient real data exists; trains the XGBoost regression and classification models; saves `temperature_model.pkl` and `rain_model.pkl` to `ml/trained_models/`; and saves evaluation metrics to `metrics.json`. The sketch below illustrates the training call.

```
# Illustrative sketch of the training step
from xgboost import XGBRegressor, XGBClassifier

reg = XGBRegressor(n_estimators=300, max_depth=6, learning_rate=0.05)
clf = XGBClassifier(n_estimators=300, max_depth=6, learning_rate=0.05)
reg.fit(X_train, y_temp_train)
clf.fit(X_train, y_rain_train)
```

6.5 Evaluation

The documentation reports the following metrics for the current models, which were trained on 1,000 synthetic samples.

Table 6.3: Documented model metrics (synthetic data)

Model	Metric	Value
Temperature	Mean absolute error	≈ 1.79 °C
Temperature	Root mean squared error	≈ 2.24
Temperature	R ²	≈ 0.71
Rain	Accuracy	$\approx 85\%$
Rain	F1 score	≈ 0.125

Interpretation

The temperature model explains roughly 71% of the variance in the synthetic data with an average error under 2 °C, which is a reasonable baseline for a first model. The rain classifier’s accuracy looks strong but its F1 score of about 0.125 shows that it detects few of the actual rain events. This is the classic signature of class imbalance: when rain is the minority class, a model that mostly predicts “no rain” attains high accuracy while being of little use. The accuracy figure should therefore not be quoted alone.

Because the data are synthetic, none of these numbers is evidence of real-world skill. They demonstrate that the pipeline works end to end; performance on real observations must be measured separately.

6.6 Grounding the LLM

To limit hallucination the LLM is used only after the factual data have been retrieved. The reply is generated from the weather observation, forecast, ML predictions and alerts that the services return, and those same objects are returned to the client alongside the text. A user can therefore compare the sentence with the numbers. Recommended additional safeguards are a system prompt that forbids stating values not present in the supplied data, and an automated check that numbers in the reply appear in the data (see Section 9.2).

6.7 Roadmap for Improving the Models

- Populate `weather_observations` with real, continuous records for target locations and retrain.
- Address class imbalance using class weights (`scale_pos_weight`), threshold tuning on a precision–recall curve, or resampling.
- Evaluate with a time-ordered (walk-forward) split so that training data always precede test data.

- Benchmark against simple baselines – persistence and the raw Open-Meteo forecast – and adopt the ML model only where it is better.
- Report precision, recall, F1 and calibration for rain, and MAE per horizon for temperature.
- Version model artefacts together with their metrics.json and training-data date range.

6.8 Analytics Presented to Users

The interface presents temperature, rainfall and monthly statistics through Chart.js charts, a predictions and statistics summary, hourly and daily forecast views, and an alerts panel. The historical endpoint aggregates ten years of data by default so that users can compare today's conditions with long-term patterns.

6.9 Summary

This chapter described the ML pipeline, reported its documented metrics with their limitations, and explained how the LLM is grounded in retrieved data. The next chapter describes testing.

CHAPTER 7: TESTING AND VALIDATION

7.1 Testing Strategy

Testing combines manual scripts, automated tests and interface checks. The repository includes manual test scripts (`test_auth.py`, `test_bangalore.py`, `test_mumbai.py` and `test_chats.py`) that exercise the running API, and `pytest` can be run from the backend folder. Interface behaviour is checked manually on desktop and mobile browsers.

Table 7.1: Testing levels

Level	Focus	Tooling
Unit	Password hashing, token creation, weather-code mapping, feature construction	pytest
Integration	API endpoints with database: registration, login, weather, chat	pytest / manual scripts
ML	Model loading, prediction shape at five horizons, metrics file	pytest / training script
System / UI	End-to-end query, charts, responsive layout	Manual, desktop and mobile

7.2 Test Environment

Tests require a running MariaDB/MySQL instance (or a test database configured through `DATABASE_URL`), a valid `GROQ_API_KEY` for chat tests and internet access for Open-Meteo. The manual scripts are run from the project root and print the responses of the endpoints they call.

```
cd weathergpt-web
python test_auth.py          # registration, login, tokens
python test_bangalore.py     # weather endpoints for Bangalore
python test_mumbai.py        # weather endpoints for Mumbai
python test_chats.py         # conversational endpoint
cd backend && python -m pytest -v
```

7.3 Test Cases

Table 7.2 lists representative test cases and the expected result. Actual pass/fail outcomes are recorded in Appendix C at the time of submission.

Table 7.2: Representative test cases

ID	Scenario	Expected result
TC-01	Register with valid details	Account created; password stored as an Argon2 hash
TC-02	Register with an existing e-mail	Rejected with a clear error
TC-03	Login with wrong password	401 Unauthorised
TC-04	Login with valid credentials	Access and refresh tokens returned; refresh cookie set
TC-05	Access /auth/me without a token	401 Unauthorised
TC-06	Refresh with a valid refresh cookie	New access token issued
TC-07	Logout	Refresh cookie cleared
TC-08	Search location “Bangalore”	Geocoding candidates returned
TC-09	Current weather for valid coordinates	Observation, forecast and predictions returned
TC-10	7-day forecast	Hourly and daily data returned
TC-11	Historical analysis default	Ten years of aggregates returned

ID	Scenario	Expected result
TC-12	Weather request with missing coordinates	Validation error, no crash
TC-13	Chat: “Will it rain in Bangalore tomorrow?”	Response with intent, entities, weather data and predictions
TC-14	Chat as anonymous user	Reply returned without login
TC-15	Chat as logged-in user	Session and messages stored and retrievable
TC-16	Save, default, edit and delete a location	Each operation succeeds for the owner only
TC-17	Search history view and clear	History returned, then emptied
TC-18	Predictions with models absent	Weather still returned; predictions omitted or flagged
TC-19	Protected route in browser when logged out	Redirect to /login
TC-20	Mobile viewport (360 px width)	Layout adapts; no horizontal scroll

7.4 Execution and Results

The manual scripts and the pytest suite are executed as shown in Section 7.2. The terminal output, the date and environment, and the number of tests passed and failed are to be pasted into Appendix C at the time of submission, together with screenshots of the application. This thesis does not assert test outcomes that have not been recorded.

7.5 Known Issues and Resolutions

The project documentation lists the following known issues. They are reproduced here with the planned resolution for each.

Table 7.3: Known issues and resolutions

Issue	Status	Resolution / plan
ML models untrained by default	Known	Run ml/train.py after populating the database; retrain on real data
IMD alerts not implemented	Known	Replace the IMDAlertService stub once IMD credentials are available
MariaDB vs PostgreSQL mismatch	Config mismatch	Verified on MariaDB; correct comments that refer to PostgreSQL
Refresh-token rotation	Partial	Invalidate the old refresh token when a new one is issued
Frontend redirect after login	Works	Add a redirect_after_login parameter
Chat history pagination	Basic	Replace hard-coded limit=50 with paging parameters
Rate limiting	Missing	Add rate limiting (for example slowapi) on auth and weather endpoints
HTTPS / production configuration	Manual	Set secure cookies, restrict CORS origins, terminate HTTPS at a proxy

7.6 Summary

This chapter presented the testing approach, representative test cases and the known issues with their resolutions. The next chapter discusses outcomes against the project objectives.

CHAPTER 8: RESULTS AND DISCUSSION

8.1 Achievement of Objectives

Table 8.1: Objectives versus outcomes

Objective	Outcome
Current, forecast and historical weather for searchable locations	Achieved: dedicated endpoints and dashboard components
Natural-language weather questions via an LLM	Achieved: Groq integration returning intent, entities and grounded data
XGBoost prediction of temperature and rain	Partially achieved: pipeline and inference in place; models trained on synthetic data
Secure authentication	Achieved: JWT access/refresh tokens, Argon2id hashing; refresh rotation partial
Personalisation	Achieved: saved locations, sessions, search history and preferences
Relational schema	Achieved: thirteen tables covering users, chat, weather and predictions
Containerised deployment and documentation	Achieved: Dockerfile, Compose configuration and production checklist
Official IMD alerts	Not achieved: service is a stub (future work)

8.2 Feature Summary of the Working System

From a user’s perspective, WeatherGPT offers a location search with autocomplete, a dashboard with current conditions, hourly and daily forecasts and charts, and a chat box that answers questions such as “Will it rain in Bangalore tomorrow?” with a sentence and the supporting data. Registered users keep their places, history and preferences. From a developer’s perspective, the system offers interactive API documentation at /docs, a health endpoint, containerised deployment and a documented training script.

8.3 Analysis of the ML Metrics

Table 8.2 restates the documented metrics with a brief reading of each. The key lesson is that the rain model’s headline accuracy overstates its usefulness.

Table 8.2: Reading the documented metrics

Metric	Value	Reading
Temperature MAE	≈ 1.79 °C	Average absolute error under 2 °C on synthetic data
Temperature RMSE	≈ 2.24	Larger than MAE, indicating some larger errors
Temperature R ²	≈ 0.71	About 71% of variance explained
Rain accuracy	$\approx 85\%$	Inflated by the dominance of the no-rain class
Rain F1	≈ 0.125	Poor detection of actual rain events

These results justify the roadmap in Section 6.7: train on real observations, correct for imbalance, evaluate with time-ordered splits and compare against the raw forecast. Until then the platform should present ML outputs as experimental and rely on the Open-Meteo forecast for primary information.

8.4 Discussion

8.4.1 Strengths

- Integrated pipeline from data source to conversational answer, with supporting data returned alongside text.
- Secure authentication using Argon2id and short-lived JWTs with an HttpOnly refresh cookie.
- Clear layered design that isolates weather, LLM and ML services.
- Typed front end whose interfaces mirror back-end schemas.
- Containerised deployment and an honest, documented list of known issues.

8.4.2 Limitations

- ML models are trained on synthetic data; the rain classifier has a low F1 score.
- Official IMD alerts are not integrated.
- No rate limiting on authentication and weather endpoints; cookies are not yet marked secure by default.
- Refresh-token rotation is partial; chat history pagination is basic.
- Answer quality depends on external services (Open-Meteo and Groq) and on internet access.
- The tests documented are largely manual; broad automated coverage remains to be added.

8.4.3 SWOT view

Table 8.3: SWOT analysis (a)

Strengths	Weaknesses
Live NWP data; conversational interface; secure accounts; modular design; open technology	Synthetic-data ML; alert stub; limited automated testing; dependence on external APIs

Table 8.4: SWOT analysis (b)

Opportunities	Threats
Real observation data for retraining; IMD integration; multilingual support; agricultural and aviation advisories	Competition from established apps; API rate or policy changes; LLM errors if grounding is weakened

8.5 Social and Economic Impact

A conversational interface lowers the barrier to using weather data, especially for people who find charts and technical products difficult. Reliable, understandable weather information supports safer travel, better planning of outdoor work and, potentially, more efficient farm operations. The stored record of observations and queries can also help identify what people most need to know.

8.6 Deployment Considerations

The project documentation provides a deployment checklist, summarised below.

- Set DEBUG=false and generate a strong SECRET_KEY of at least 32 characters.
- Use HTTPS and set secure=True on cookies; configure production CORS origins.
- Use a managed database such as RDS or Cloud SQL, and configure backups for MariaDB.

- Keep the Groq key in a secrets manager.
- Train the ML models on real historical data; configure the IMD API if targeting India.
- Add rate limiting, log aggregation and health-check monitoring.

CHAPTER 9: CONCLUSION AND FUTURE SCOPE

9.1 Conclusion

This thesis addressed the problem that weather information is abundant but scattered and hard for ordinary users to use. WeatherGPT was designed and implemented as a full-stack conversational weather platform that combines Open-Meteo forecasts, XGBoost predictions and a Groq-hosted LLM behind a FastAPI service and a React interface, with secure authentication and personalisation.

The work demonstrates that a small team using open-source tools and free-tier services can deliver an integrated, containerised system that answers questions in natural language while returning the data behind each answer. It also shows the importance of honest evaluation: the rain classifier's high accuracy masks a low F1 score, and the models are trained on synthetic data. These limitations, and the missing IMD integration and rate limiting, define the next steps.

9.2 Future Scope

- Retrain the temperature and rain models on real, continuous observations with time-ordered evaluation and imbalance handling.
- Integrate official IMD alerts by replacing the stub service.
- Add grounding checks that verify numbers in LLM replies against the retrieved data, and support regional-language replies.
- Add rate limiting, complete refresh-token rotation, secure cookies and full HTTPS configuration.
- Increase automated test coverage, including CI runs of pytest on every commit.
- Add weather-triggered notifications for saved locations.
- Add agricultural, travel and aviation advisories built on the forecast and predictions.
- Build native mobile applications and offline-tolerant views.
- Add radar and satellite layers and a map view.

9.3 Closing Remarks

WeatherGPT shows that technology can shorten the distance between raw atmospheric data and a person's decision. With real training data, official alert integration and continued hardening, the platform can grow from a well-structured prototype into a dependable everyday tool.

REFERENCES

1. S. Ramírez, “FastAPI Documentation,” <https://fastapi.tiangolo.com>.
2. SQLAlchemy Project, “SQLAlchemy Documentation,” <https://docs.sqlalchemy.org>.
3. Meta Open Source, “React Documentation,” <https://react.dev>.
4. Vite Contributors, “Vite Documentation,” <https://vitejs.dev>.
5. Open-Meteo, “Open-Meteo Weather Forecast and Geocoding API Documentation,” <https://open-meteo.com/en/docs>.
6. Groq, Inc., “GroqCloud Documentation,” <https://console.groq.com/docs>.
7. T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2016, pp. 785–794.
8. F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.
9. M. Jones, J. Bradley and N. Sakimura, “JSON Web Token (JWT),” RFC 7519, Internet Engineering Task Force, 2015.
10. A. Biryukov, D. Dinu, D. Khovratovich and S. Josefsson, “Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications,” RFC 9106, IETF, 2021.
11. P. Bauer, A. Thorpe and G. Brunet, “The quiet revolution of numerical weather prediction,” Nature, vol. 525, pp. 47–55, 2015.
12. ECMWF, “IFS Documentation,” <https://www.ecmwf.int/en/publications/ifs-documentation>.
13. India Meteorological Department, <https://mausam.imd.gov.in>.
14. Chart.js Contributors, “Chart.js Documentation,” <https://www.chartjs.org/docs>.
15. pytest-dev, “pytest Documentation,” <https://docs.pytest.org>.
16. Docker Inc., “Docker Compose Documentation,” <https://docs.docker.com/compose>.

APPENDIX A: INSTALLATION AND EXECUTION GUIDE

A.1 Prerequisites

- Python 3.12 with pip and venv
- Node.js 18 or later
- MariaDB / MySQL 10.6 or later running on port 3306
- A free Groq API key from console.groq.com

A.2 Steps

```
# 1. Backend
cd weathergpt-web/backend
python3.12 -m venv venv312 && source venv312/bin/activate
pip install -r requirements.txt
cp .env.example .env      # set DATABASE_URL, GROQ_API_KEY, SECRET_KEY
python -c "from backend.database import init_db; init_db()"
python -m uvicorn backend.main:app --host 0.0.0.0 --port 8000 --reload

# 2. Frontend
cd weathergpt-web/frontend
npm install
npm run dev

# 3. Optional: train ML models
cd weathergpt-web && python ml/train.py
```

A.3 Access URLs

Table A.1: Local URLs

Component	URL
Web application	http://localhost:5173
API base	http://localhost:8000
Interactive API documentation	http://localhost:8000/docs
Health check	http://localhost:8000/health

APPENDIX B: SAMPLE API REQUESTS AND RESPONSES

B.1 Login

```
POST /api/auth/login
{ "email": "user@example.com", "password": "password123" }

200 OK
{ "access_token": "eyJ...", "refresh_token": "eyJ...",
  "token_type": "bearer", "expires_in": 1800 }
```

B.2 Current weather

```
GET /api/weather/current?latitude=12.9716&longitude=77.5946&location_name=Bangalore

{ "location": { "name": "Bangalore", "latitude": 12.9716, ... },
  "observation": { "temperature": 28.5, "humidity": 65,
    "weather_description": "Partly cloudy", ... },
  "forecast": [...], "alerts": [],
  "predictions": { "temperature": { "1h": {...}, ..., "24h": {...} },
    "rain": { "1h": {...}, ..., "24h": {...} } } }
```

B.3 Chat

```
POST /api/chat
{ "message": "Will it rain in Bangalore tomorrow?",
  "latitude": 12.9716, "longitude": 77.5946,
  "location_name": "Bangalore", "language": "en" }

{ "response": "...", "intent": "rain_probability",
  "entities": { "location": { "name": "Bangalore", ... } },
  "weather_data": {...}, "predictions": {...}, "alerts": [],
  "session_id": "uuid", "response_time_ms": 1250 }
```

APPENDIX C: TEST EXECUTION RECORD

Paste the terminal output of the manual test scripts and of the pytest run, and screenshots of the application, here at the time of submission. Suggested screenshots: (1) landing page; (2) registration and login; (3) dashboard with current weather; (4) forecast and charts; (5) chat answering a rain question; (6) saved locations; (7) interactive API documentation at /docs; (8) ML metrics file.

Table C.1: Test run record

Item	Details to record
Commands	python test_auth.py; test_bangalore.py; test_mumbai.py; test_chats.py; python -m pytest -v
Date and environment	
Number of tests passed / failed	
Notes	

APPENDIX D: DOCKER COMPOSE CONFIGURATION

The following Compose file, taken from the project documentation, runs the database, backend and frontend together. Values in braces are read from the environment.

```
version: '3.8'
services:
  db:
    image: mariadb:10.11
    environment:
      MYSQL_DATABASE: weathergpt
      MYSQL_USER: weathergpt
      MYSQL_PASSWORD: weathergpt
      MYSQL_ROOT_PASSWORD: rootpass
    ports: ["3306:3306"]
    volumes: ["db_data:/var/lib/mysql"]
  backend:
    build: ./backend
    ports: ["8000:8000"]
    environment:
      DATABASE_URL: mysql+pymysql://weathergpt:weathergpt@db:3306/weathergpt
      GROQ_API_KEY: ${GROQ_API_KEY}
      SECRET_KEY: ${SECRET_KEY}
    depends_on: [db]
  frontend:
    build: ./frontend
    ports: ["5173:5173"]
    environment:
      VITE_API_URL: http://localhost:8000/api
    depends_on: [backend]
volumes:
  db_data:
```

The default passwords above are for local demonstration only and must be changed before any real deployment.